Contents

Table of Figures

Table of Table

Retail Sales Analyzation

1. Introduction

Retail business are the most common type of business available in any country. Retail businesses generates vast amount of transactional data every day, every week and every month. These data could be customer's demographics, purchasing behavior, product categories, and revenue streams. Such amount of data needs to be managed very well so that any retail business could identify sales trends, understand customer's preference, detect anomalies and ultimately guiding strategic decision-making. The project **"Retails Sales Analyzation",** mainly focusses on dataset of thousands retail transactions to analyze the sales of the retail shop.

The dataset contains the information such as transaction date, customer ID, age, gender, product category, quantity purchased, price per unit, and total amount spent. For the analyzation of the sales trend, we will be applying statistical methods, visualization techniques, and machine learning models. The analyzation aims to uncover meaningful insights into sales performance and customer behavior.

1.1 Problem Statement

Retail businesses operate in highly competitive environments where customers preference, product demand, and sales performance fluctuate constantly. The daily data collected from the retail business or shops are in a raw form. The valuable insights cannot be extracted from unstructured or raw data. The limited features of the dataset could be a huge backlash for a business for clear visualization of the dataset and draw meaningful informations. It is very hard to transform these raw data into actionable insights. Some of the problem statements are mentioned follows:

- Unstructured Data
- Outlier Transactions
- Feature Limitations
- Business Insight Gap

1.2 Project as Solution

The upper mentioned problems and challenges like unstructured categorical data, outlier transactions, and limited explanatory features requires a structures analytical approach. The project provides a comprehensive solution for these problems by combining data preprocessing, explanatory data analysis, feature engineering, and predictive modeling into unified workflow.

Solution Approach of the Project

- **Data Cleaning & Preparation**
  - Converted date fields into proper datetime format.
  - Verified absence of missing values.
  - Standardized numerical ranges and encoded categorical variables (e.g., gender encoding, age grouping).
- **Exploratory Data Analysis (EDA)**
  - Visualized demographic distributions (gender, age groups).
  - Analyzed product category popularity and revenue contributions.
  - Identified daily and monthly sales trends to highlight seasonality.
- **Outlier Detection**
  - Applied Z-score and IQR methods to flag extreme transactions.
  - Attempted advanced detection using Local Outlier Factor (LOF), learning the importance of reshaping data for multidimensional analysis.
- **Feature Engineering**
  - Created derived attributes such as weekend indicators, price–quantity interactions, and product popularity scores.
  - Enhanced dataset richness to improve predictive modeling capacity.
- **Predictive Modeling**
  - Implemented Linear Regression to estimate total sales amounts.
  - Identified preprocessing gaps (string variables not encoded), highlighting the need for one-hot encoding and categorical handling.
  - Established a foundation for future use of advanced models (Random Forest, Gradient Boosting).

1.3 Aim and Objectives of the Project

Aims

The aim of this project is to analyze retail sales data to uncover the hidden trends and patterns of the business for developing a structured workflow for data preprocessing, visualization and predictive modelling.

Objectives

- To understand the dataset and business model.
- To visualize the dataset using various data visualization techniques.
- To detect the outliers and anomalies.
- To add necessary features using feature engineering.

2. Data Understanding

2.1 Dataset Overview

The dataset chosen for the project is extracted from a retail business shop. The dataset contains almost 1000s plus retail transaction data, each representing a purchase event. The dataset includes both demographic and transactional details.

• **Entries:** 1000 rows	• **Columns:**
• **Data Types:** 5 integer fields, 4 object (string) fields	• **Missing Values:** None detected

Table 1: Dataset Features

2.2 Variables in the Dataset

- **Transaction ID (int):** Unique identifier for each transaction.
- **Date (datetime):** Transaction date, later converted to datetime for time-series analysis.
- **Customer ID (object):** Unique identifier for each customer.
- **Gender (object):** Male or Female.

- **Age (int):** Customer's age, ranging from 18 to 64.
- **Product Category (object):** Clothing, Electronics, or Beauty.
- **Quantity (int):** Number of units purchased (range: 1–4).
- **Price per Unit (int):** Price of a single unit (range: 25–500).
- **Total Amount (int):** Transaction value (Quantity × Price per Unit).

2.3 Descriptive Statistics

- **Age:** Mean = 41.39, Std = 13.68, Range = 18–64.
- **Quantity:** Mean = 2.51, Std = 1.13, Range = 1–4.
- **Price per Unit:** Mean = 179.89, Std = 189.68, Range = 25–500.
- **Total Amount:** Mean = 456, Std = 560, Range = 25–2000.

2.4 Distribution Insights

- **Gender:** Balanced distribution (Female 51%, Male 49%).
- **Product Category:** Clothing (35%), Electronics (34%), Beauty (31%).
- **Age Groups:** Majority of customers fall between 20–60 years.

2.5 Data Quality Observations

- No missing values or null entries.
- Outliers detected in Total Amount (transactions at 2000).
- Categorical variables (Customer ID, Product Category, Gender) require encoding for modeling.
- Date field conversion enables extraction of Year, Month, Day, and DayOfWeek for temporal analysis.

2.6 Tools

- **IDE:** Jupyter Notebook or Google Collab or VS code integrated with Jupyter Notebook.
- **Libraries:** pandas, numpy, matplotlib, seaborn, scipy, scikit-learn.

3. Data Preparation

3.1 Import Dataset

In this step, the collected dataset is loaded to the IDE using the pandas. For reading the csv file read_csv function is used using the pandas library as pd. All the required necessary library like pandas, numpy and scipy.stats have successfully imported to do the required calculations.

```
import pandas as pd

sales_data = pd.read_csv('retail_sales_dataset.csv')
print(sales_data.head())
```

0.0s Python

```
   Transaction ID       Date Customer ID  Gender  Age Product Category  \
0               1 2023-11-24     CUST001    Male   34           Beauty
1               2 2023-02-27     CUST002  Female   26         Clothing
2               3 2023-01-13     CUST003    Male   50      Electronics
3               4 2023-05-21     CUST004    Male   37         Clothing
4               5 2023-05-06     CUST005    Male   30           Beauty

   Quantity  Price per Unit  Total Amount
0         3              50           150
1         2             500          1000
2         1              30            30
3         1             500           500
4         2              50           100
```

Figure 1: Loading Dataset

3.2 Checking Characteristics of the Dataset

In this step, we are checking the extra features already the dataset have. For this we will be seeing its shape, info null values and other.

```
print(sales_data.info())
print(sales_data.describe())
print(sales_data.isnull().sum())
```

[22] 0.0s Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    1000 non-null   int64
 1   Date              1000 non-null   object
 2   Customer ID       1000 non-null   object
 3   Gender            1000 non-null   object
 4   Age               1000 non-null   int64
 5   Product Category  1000 non-null   object
 6   Quantity          1000 non-null   int64
 7   Price per Unit    1000 non-null   int64
 8   Total Amount      1000 non-null   int64
dtypes: int64(5), object(4)
memory usage: 70.4+ KB
None
       Transaction ID          Age     Quantity  Price per Unit  Total Amount
count     1000.000000  1000.00000  1000.000000     1000.000000   1000.000000
mean       500.500000    41.39200     2.514000      179.890000    456.000000
std        288.819436    13.68143     1.132734      189.681356    559.997632
min          1.000000    18.00000     1.000000       25.000000     25.000000
25%        250.750000    29.00000     1.000000       30.000000     60.000000
50%        500.500000    42.00000     3.000000       50.000000    135.000000
75%        750.250000    53.00000     4.000000      300.000000    900.000000
...
Quantity            0
Price per Unit      0
Total Amount        0
dtype: int64
```

Figure 2: Checking out Features of the Dataset

3.3 Identifying Genders of the Customers

In a retail sales analysis, it is very crucial to identify whether the customer is male or female. For this identification, we will be using value counts function as shown below in the image.

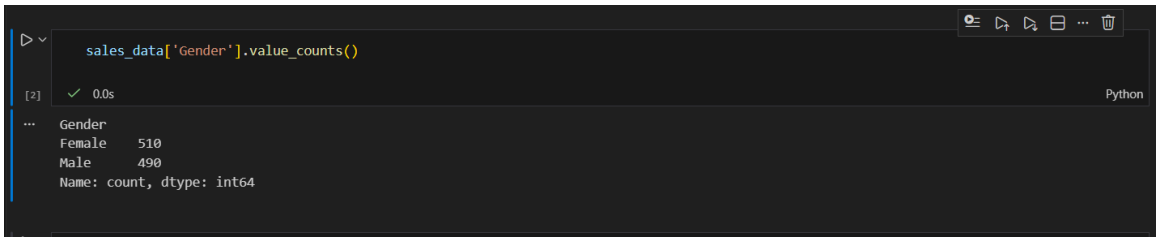


Figure 3: Identifying Genders

3.4 Identifying the Products Types

In a retail sales analysis, it is very crucial to identify the category of product. Here in the dataset we have multiple category of the products. For this identification, we will be using value counts function as shown below in the image.

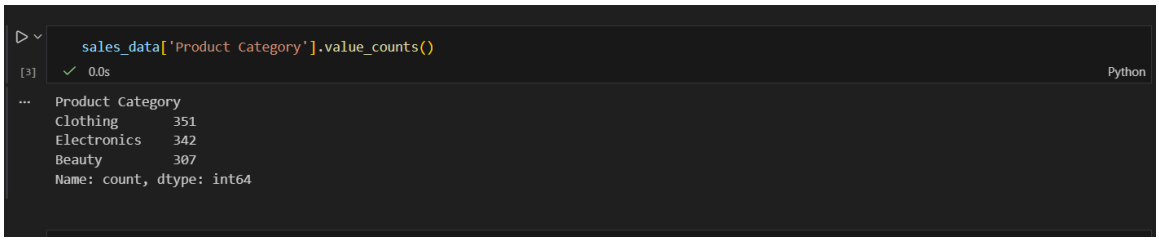


Figure 4: Identifying the Products Types

4. Explanatory Data Analysis

Exploratory Data Analysis (EDA) is an important step in data science and data analytics as it visualizes data to understand its main features, find patterns and discover how different parts of the data are connected. There are several types of visualizations that are commonly used in EDA using Python, including:

- **Bar charts**: Used to show comparisons between different categories.
- **Line charts**: Used to show trends over time or across different categories.
- **Pie charts**: Used to show proportions or percentages of different categories.
- **Histograms**: Used to show the distribution of a single variable.

4.1 Daily Sales Visualization

The daily sales analyzation of the retails shop can be done using visualization technique. For this daily sales analyzation visualization, line graph had been used.

For the line graph to be displayed, date column and total sum amounts have been used.

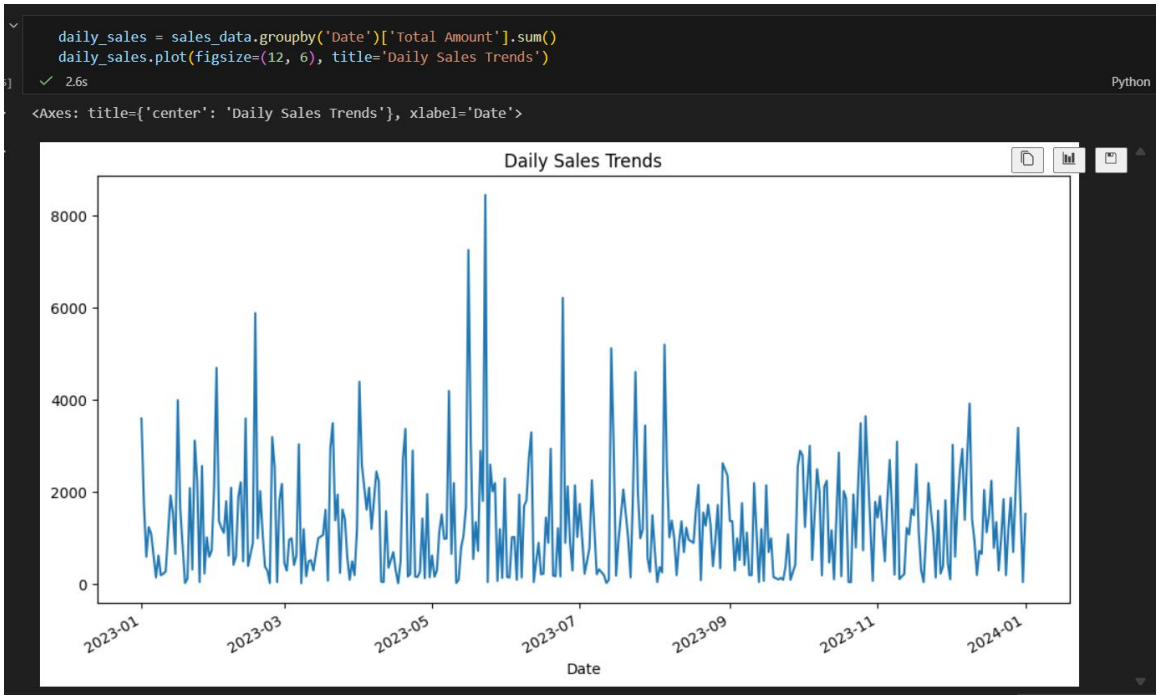


Figure 5: Daily Sales Visualization

4.2 Total Sales by Product Category

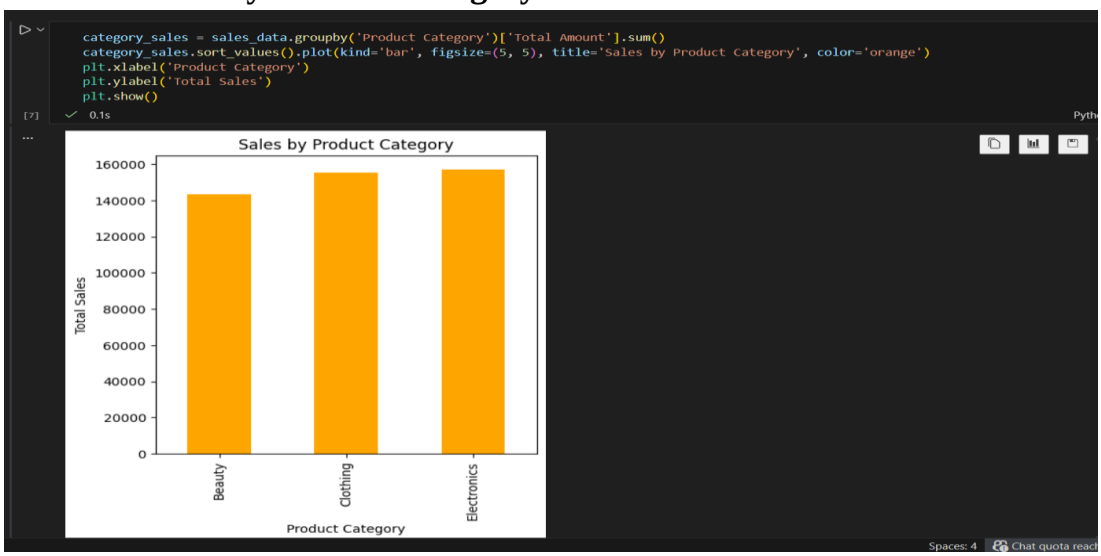


Figure 6: Sales by Product Category

4.3 Relationship Between Price, Quantity and Total Amount

```
import seaborn as sns
# Pairplot to visualize relationships
sns.pairplot(sales_data[['Quantity', 'Price per Unit', 'Total Amount']])
plt.show()
```

[9] ✓ 12.8s Python

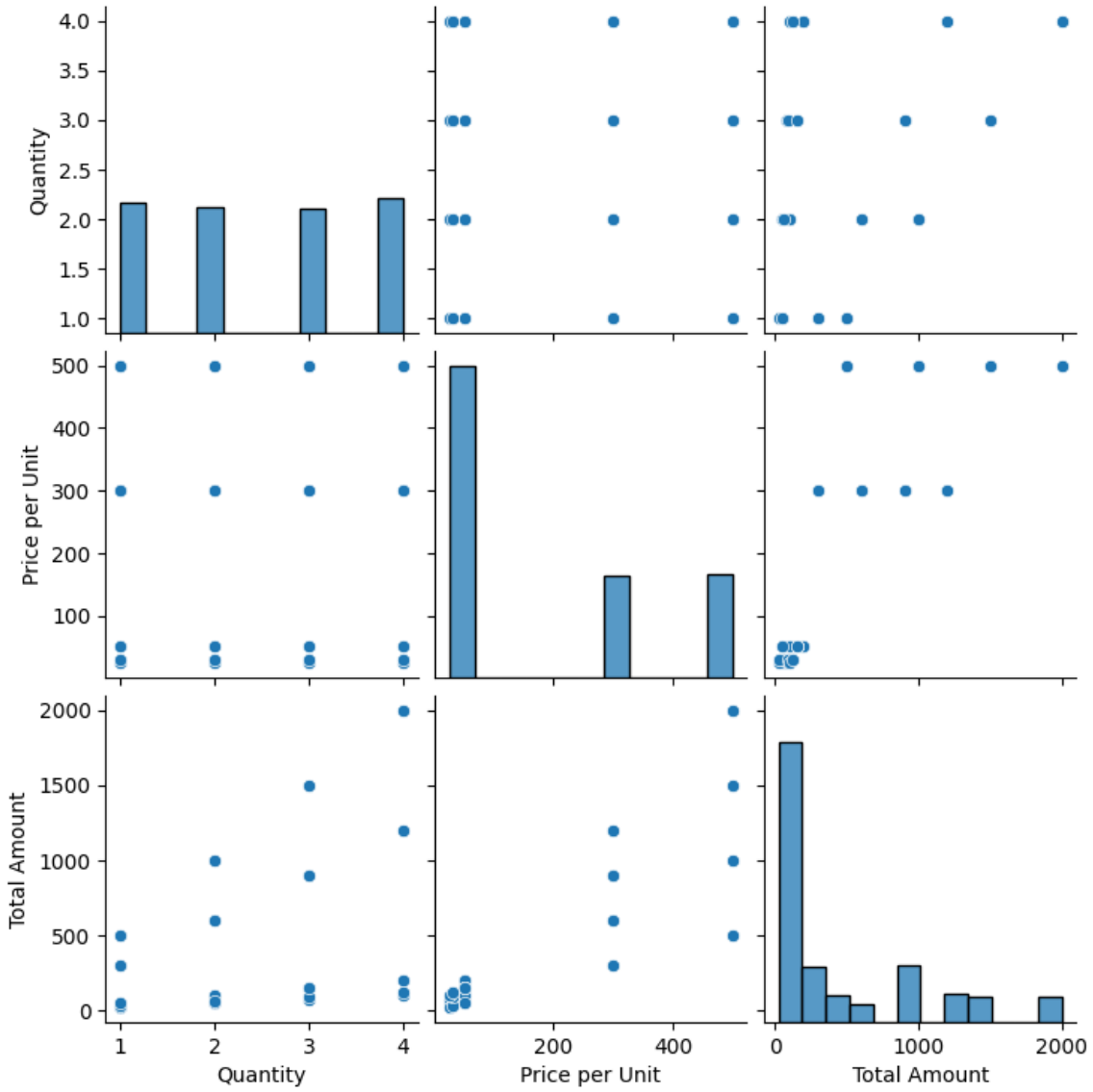


Figure 7: Relationship between Quantity, Price and Total Amount

4.4 Detecting Outliers

```
from scipy.stats import zscore

sales_data['Z-score'] = zscore(sales_data['Total Amount'])

outliers = sales_data[abs(sales_data['Z-score'])>2]

print(outliers)
```

[94] ✓ 0.0s

```
     Transaction ID       Date Customer ID  Gender  Age Product Category  \
14               15 2023-01-16     CUST015  Female   42      Electronics
64               65 2023-12-05     CUST065    Male   51      Electronics
71               72 2023-05-23     CUST072  Female   20      Electronics
73               74 2023-11-22     CUST074  Female   18           Beauty
88               89 2023-10-01     CUST089  Female   55      Electronics
92               93 2023-07-14     CUST093  Female   35           Beauty
108             109 2023-10-18     CUST109  Female   34      Electronics
117             118 2023-05-16     CUST118  Female   30      Electronics
123             124 2023-10-27     CUST124    Male   33         Clothing
138             139 2023-12-15     CUST139    Male   36           Beauty
151             152 2023-02-28     CUST152    Male   43      Electronics
154             155 2023-05-17     CUST155    Male   31      Electronics
156             157 2023-06-24     CUST157    Male   62      Electronics
165             166 2023-04-02     CUST166    Male   34         Clothing
252             253 2023-08-31     CUST253  Female   53         Clothing
256             257 2023-02-19     CUST257    Male   19           Beauty
268             269 2023-02-01     CUST269    Male   25         Clothing
280             281 2023-05-23     CUST281  Female   29           Beauty
341             342 2023-10-24     CUST342  Female   43         Clothing
411             412 2023-09-16     CUST412  Female   19      Electronics
415             416 2023-02-17     CUST416    Male   53      Electronics
419             420 2023-01-23     CUST420  Female   22         Clothing
446             447 2023-07-06     CUST447    Male   22           Beauty
475             476 2023-08-29     CUST476  Female   27         Clothing
...
```

Figure 8: Detecting Outliers

5. Splitting Dataset and Model Selection

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size= 0.2, random_state= 42)
```

[108] ✓ 0.0s Python

```
categorical_cols = x.select_dtypes(include=['object', 'category']).columns.tolist()
numeric_cols = [col for col in x.columns if col not in categorical_cols]

# Preprocessing: OneHotEncode categorical variables, pass through numeric unchanged
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols),
        ('num', 'passthrough', numeric_cols)
    ]
    )
```

[109] ✓ 0.0s Python

```
from sklearn.ensemble import RandomForestRegressor

regressor = RandomForestRegressor(n_estimators=200, random_state=42)
model = Pipeline(steps=[ ('preprocessor', preprocessor), ('regressor', regressor) ])
```

[113] ✓ 0.8s Python

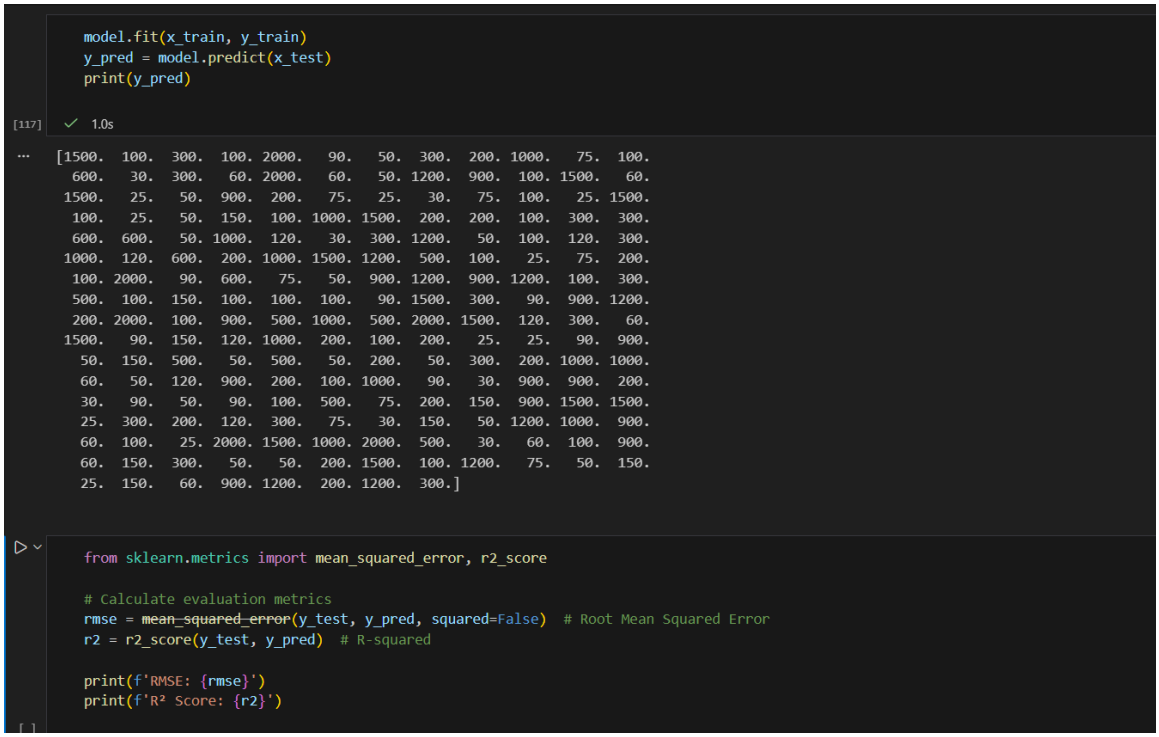
```
print(x.dtypes)

# Identify columns that are not numeric
non_numeric_columns = x.select_dtypes(include=['object']).columns
print(f"Non-numeric columns: {non_numeric_columns}")
```

[ ] Python

Figure 10: Splitting Dataset and Model Selection

6. Model Testing



```
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print(y_pred)
```

[117] ✓ 1.0s

```
[1500.  100.  300.  100. 2000.   90.   50.  300.  200. 1000.   75.  100.
  600.   30.  300.   60. 2000.   60.   50. 1200.  900.  100. 1500.   60.
 1500.   25.   50.  900.  200.   75.   25.   30.   75.  100.   25. 1500.
  100.   25.   50.  150.  100. 1000. 1500.  200.  200.  100.  300.  300.
  600.  600.   50. 1000.  120.   30.  300. 1200.   50.  100.  120.  300.
 1000.  120.  600.  200. 1000. 1500. 1200.  500.  100.   25.   75.  200.
  100. 2000.   90.  600.   75.   50.  900. 1200.  900. 1200.  100.  300.
  500.  100.  150.  100.  100.  100.   90. 1500.  300.   90.  900. 1200.
  200. 2000.  100.  900.  500. 1000.  500. 2000. 1500.  120.  300.   60.
 1500.   90.  150.  120. 1000.  200.  100.  200.   25.   25.   90.  900.
   50.  150.  500.   50.  500.   50.  200.   50.  300.  200. 1000. 1000.
   60.   50.  120.  900.  200.  100. 1000.   90.   30.  900.  900.  200.
   30.   90.   50.   90.  100.  500.   75.  200.  150.  900. 1500. 1500.
   25.  300.  200.  120.  300.   75.   30.  150.   50. 1200. 1000.  900.
   60.  100.   25. 2000. 1500. 1000. 2000.  500.   30.   60.  100.  900.
   60.  150.  300.   50.   50.  200. 1500.  100. 1200.   75.   50.  150.
   25.  150.   60.  900. 1200.  200. 1200.  300.]
```

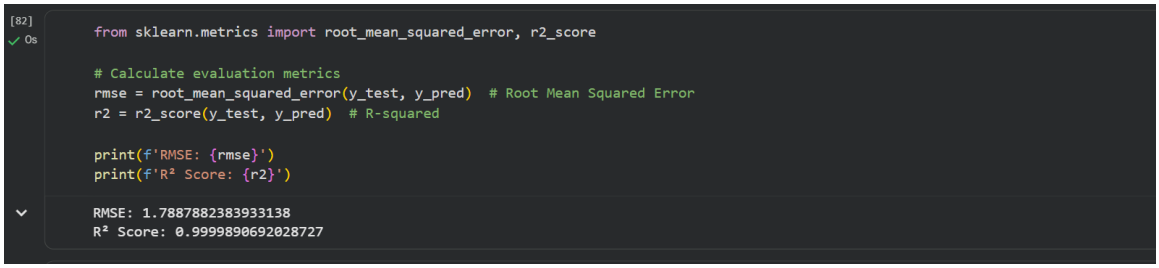
```
from sklearn.metrics import mean_squared_error, r2_score

# Calculate evaluation metrics
rmse = mean_squared_error(y_test, y_pred, squared=False)  # Root Mean Squared Error
r2 = r2_score(y_test, y_pred)  # R-squared

print(f'RMSE: {rmse}')
print(f'R² Score: {r2}')
```

Figure 11: Model Testing

7. Model Evaluation



[82] ✓ 0s

```
from sklearn.metrics import root_mean_squared_error, r2_score

# Calculate evaluation metrics
rmse = root_mean_squared_error(y_test, y_pred)  # Root Mean Squared Error
r2 = r2_score(y_test, y_pred)  # R-squared

print(f'RMSE: {rmse}')
print(f'R² Score: {r2}')

RMSE: 1.7887882383933138
R² Score: 0.9999890692028727
```

Figure 12: Model Evaluation

7.1 Interpretation of Results

The model shows that the RMSE and R^2 is 1.79 and 0.99999 which means:

The model's average prediction error is less than 2 units of currency. Considering transaction values range from 25 to 2000, this error is negligible. It means the model is predicting sales amounts with extremely high accuracy.

R^2 Score ≈ 0.99999

The model explains almost all of the variance in the target variable (Total Amount). In other words, the features used (especially Quantity and Price per Unit) are nearly perfect predictors of the sales amount.

8. Conclusion

The predictive modeling stage of the project demonstrates that retail sales amounts can be predicted with near-perfect accuracy when both unit price and quantity are available as features. The Random Forest model achieved an RMSE of 1.79 and an R^2 of 0.99999, confirming that the dataset's structure strongly determines the target variable.

This outcome highlights two key insights:

- **Deterministic relationships**: Some business metrics (like transaction totals) are direct mathematical functions of input features. Predictive models will naturally achieve near-perfect accuracy in such cases.
- **Future directions**: To extract deeper business insights, future modeling should focus on less deterministic targets — for example, predicting product category choice, customer segmentation, or forecasting monthly sales trends. These tasks introduce uncertainty and allow machine learning to uncover patterns beyond simple arithmetic.